introduction to cryptography

---

1. Symmetric vs Asymmetric Encryption

Symmetric encryption

- Uses **one single secret key** for both encryption and decryption.
- The sender and receiver must **both know the same key**.
- Very **fast and efficient**, especially for large files.

Example:
AES, DES (older, insecure)

Problem:
If someone intercepts the key while it's being shared, security is broken.

Asymmetric encryption

- Uses **two keys**:
  - **Public key** → shared with everyone
  - **Private key** → kept secret
- Data encrypted with one key can **only be decrypted with the other**.

Example:
RSA, ECC

Advantage:
No need to share secret keys beforehand.

Disadvantage:
Slower than symmetric encryption.

In practice:
Most systems use **both**: asymmetric encryption to exchange keys, then symmetric encryption for data transfer.

---

2. Encrypt Files Using AES

AES (Advanced Encryption Standard) is the most widely used symmetric encryption algorithm today.

- Supports **128, 192, and 256-bit keys**
- Considered **very secure** when used correctly

- Used in disk encryption, Wi-Fi security, VPNs, and cloud storage

How it works (conceptually):

1. You choose a password or secret key.
2. AES scrambles the file data using multiple mathematical rounds.
3. Without the key, the file looks like random noise.

Why AES matters:
Even governments and banks trust AES for protecting sensitive data.

---

3. Generate RSA Keys

RSA is an asymmetric encryption algorithm.

When generating RSA keys:

- You create a **key pair**:
  - Public key → used to encrypt data
  - Private key → used to decrypt data
- Keys are based on **large prime numbers**, making them hard to break.

Typical RSA key sizes:

- 2048 bits (standard minimum today)
- 3072 or 4096 bits (more secure, slower)

Common uses:

- Secure login systems
- Email encryption
- HTTPS certificates

You never share your **private key** — ever.

---

4. Understand Digital Signatures

A **digital signature** proves:

- **Who sent the data**
- **That it wasn't modified**
- **That the sender cannot deny sending it**

How it works:

1. A file is hashed.
2. The hash is encrypted using the sender's **private key**.
3. Anyone can verify it using the sender's **public key**.

What digital signatures provide:

- Authentication
- Integrity
- Non-repudiation

Real-world examples:

- Software updates
- Legal documents
- Blockchain transactions

---

5. Hash Files and Verify Integrity

A **hash function**:

- Takes any input and produces a **fixed-length output**
- Is **one-way** (cannot be reversed)
- Changes completely if even **one bit** of data changes

Common hash algorithms:

- SHA-256 (secure, widely used)
- SHA-1 (deprecated)
- MD5 (broken, not secure)

Why hashing matters:

- Verifying file integrity
- Storing passwords securely
- Detecting tampering

If two hashes match, the files are **identical**.

---

6. Compare Encryption Algorithms

Algorithm	Type	Speed	Security	Use Case
AES	Symmetric	Very fast	Very high	File encryption, VPNs
RSA	Asymmetric	Slow	High	Key exchange, HTTPS

Algorithm	Type	Speed	Security	Use Case
ECC	Asymmetric	Faster than RSA	Very high	Mobile devices
DES	Symmetric	Fast	Broken	Legacy only

Key takeaway:
No single algorithm does everything best — they're chosen based on the problem.

---

7. Real-World Usage (HTTPS, VPN)

HTTPS

- Uses RSA/ECC to exchange keys
- Uses AES to encrypt web traffic
- Protects logins, payments, and personal data

VPN

- Encrypts all network traffic
- Uses AES for data encryption
- Uses certificates or keys for authentication

Other uses:

- Secure messaging apps (Signal, WhatsApp)
- Disk encryption (BitLocker, FileVault)
- Cloud storage security

---

8. Documentation of Findings (Summary)

- Symmetric encryption is fast but requires secret key sharing.
- Asymmetric encryption solves key exchange problems.
- AES is the gold standard for encrypting data.
- RSA enables secure communication over insecure networks.
- Hashing ensures data integrity.
- Digital signatures prove authenticity.
- Modern security systems combine multiple cryptographic methods.
